Pydantic vs Native Python Class (OOP)

1. Apa Itu Pydantic?

Pydantic adalah library Python untuk mendefinisikan struktur data, melakukan validasi tipe data, mengelola data, dan melakukan serialisasi.

Pydantic menggunakan Python Class dengan mewarisi `BaseModel`.

Instalasi:

```
None
pip install pydantic
```

2. Perbandingan Native Class vs Pydantic

A. Native Python Class

```
None
class User:
    def __init__(self, name, age):
        self.name = name
        self.age = age

user = User("Akmal", "25")

print(user.name)
print(user.age)
```

Native Class tidak otomatis memvalidasi tipe data. Variabel `age` tetap bisa menerima string meskipun secara konsep seharusnya berupa integer.

B. Pydantic BaseModel

```
None
from pydantic import BaseModel

class User(BaseModel):
    name: str
```

```
    age: int

user = User(name="Akmal", age="25")

print(user.name)
print(user.age)
print(type(user.age))
```

Output:

```
None
Akmal
25
<class 'int'>
```

Pydantic otomatis mengonversi string `"25"` menjadi integer `25` dalam mode validasi standar.

Jika data tidak dapat dikonversi, Pydantic akan menghasilkan `ValidationError`.

3. Perbandingan Fitur

Aspek	Native Class	Pydantic
Membuat objek	Ya	Ya
Type hints	Ya	Ya
Validasi otomatis	Tidak	Ya
Type conversion	Manual	Otomatis (default)
Custom validation	Manual	Built-in validators
JSON serialization	Manual	Built-in
JSON Schema	Manual	Built-in
Dependency	Python native	Library tambahan

4. Mengapa Menggunakan Pydantic?

1. Automatic Data Validation

Memastikan data sesuai dengan tipe dan aturan yang ditentukan.

2. Data Type Conversion

Mengonversi data ke tipe yang sesuai selama proses validasi, jika memungkinkan.

3. Serialization

Mengubah objek menjadi dictionary atau JSON dengan mudah.

```
None
print(user.model_dump())
print(user.model_dump_json())
```

4. Custom Validation

Memungkinkan kita menentukan aturan validasi tambahan menggunakan `field_validator`.

```
None
from pydantic import BaseModel, field_validator

class User(BaseModel):
    name: str
    age: int

    @field_validator("age")
    @classmethod
    def validate_age(cls, value):
        if value < 18:
            raise ValueError("Age must be at least 18")
        return value
```

5. Kapan Menggunakan Masing-Masing?

Kebutuhan	Rekomendasi
Membuat objek dan business logic	Native Class
Membuat struktur data API	Pydantic
Validasi input pengguna	Pydantic
Data processing dan ETL	Keduanya

Complex object-oriented application	Native Class
JSON request dan response	Pydantic
Data configuration	Pydantic

6. Kesimpulan

Pydantic bukan pengganti OOP, melainkan tools yang melengkapi konsep OOP.

- Gunakan Native Class ketika membutuhkan objek, method, inheritance, dan business logic.
- Gunakan Pydantic ketika membutuhkan struktur data yang tervalidasi, konsisten, dan mudah diserialisasi.
- Gunakan keduanya secara bersamaan ketika membangun aplikasi Python yang kompleks.